

TYPES OF INHERITANCE

1. Single Inheritance

Problem Statement

Write a Java program to demonstrate Single Inheritance, where a Dog class inherits the properties and methods of an Animal class.

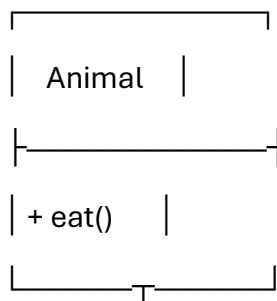
Steps

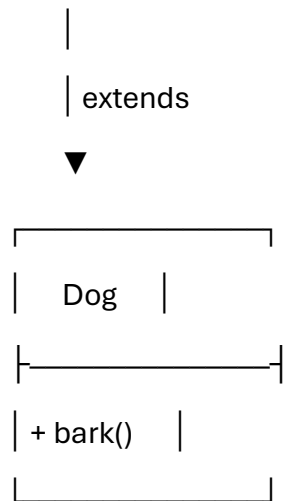
- Create a parent class Animal.
- Define an eat() method inside Animal.
- Create a child class Dog that extends Animal.
- Define a bark() method inside Dog.
- Create an object of Dog.
- Call both the inherited eat() method and the bark() method.
- Display the output.

Algorithm

1. Start.
2. Define class Animal with method eat().
3. Define class Dog using extends Animal.
4. Define method bark() inside Dog.
5. Create a Dog object in the main() method.
6. Call dog.eat().
7. Call dog.bark().
8. Stop.

UML Diagram





Code

// File: SingleInheritance.java

// Parent class

```
class Animal {
    void eat() {
        System.out.println("Animal eats food.");
    }
}
```

// Dog extends Animal.

// This is an example of Single Inheritance because

// one child class inherits from one parent class.

```
class Dog extends Animal {
    void bark() {
        System.out.println("Dog barks.");
    }
}
```

```
public class SingleInheritance {  
    public static void main(String[] args) {  
  
        Dog dog = new Dog();  
  
        // Method inherited from Animal  
        dog.eat();  
  
        // Method belonging to Dog  
        dog.bark();  
    }  
}
```

Output

Animal eats food.

Dog barks.

2. Multiple Inheritance

Problem Statement

Write a Java program to demonstrate Multiple Inheritance using interfaces, where a Duck class implements both Flyable and Swimmable interfaces.

Steps

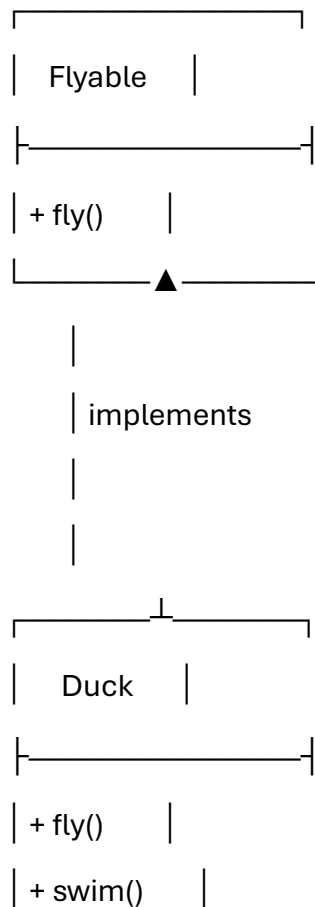
- Create an interface Flyable.
- Declare the fly() method.
- Create another interface Swimmable.
- Declare the swim() method.
- Create a Duck class implementing both interfaces.

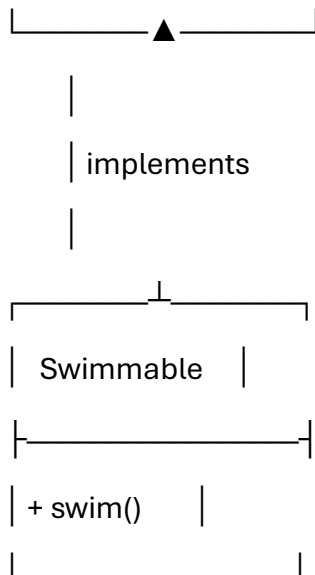
- Implement both methods in Duck.
- Create a Duck object.
- Call fly() and swim().

Algorithm

1. Start.
2. Create interface Flyable with fly().
3. Create interface Swimmable with swim().
4. Create class Duck implementing both interfaces.
5. Define the fly() method.
6. Define the swim() method.
7. Create a Duck object.
8. Call duck.fly().
9. Call duck.swim().
10. Stop.

UML Diagram





Code

// File: MultipleInheritance.java

// First interface

```
interface Flyable {  
    void fly();  
}
```

// Second interface

```
interface Swimmable {  
    void swim();  
}
```

// Duck implements two interfaces.

// Java does not support multiple inheritance through classes,

// but multiple inheritance can be achieved using interfaces.

```
class Duck implements Flyable, Swimmable {
```

```

@Override
public void fly() {
    System.out.println("Duck is flying in the sky.");
}

@Override
public void swim() {
    System.out.println("Duck is swimming in the water.");
}
}

public class MultipleInheritance {
    public static void main(String[] args) {

        Duck duck = new Duck();

        // Method implemented from Flyable
        duck.fly();

        // Method implemented from Swimmable
        duck.swim();
    }
}

```

Output

Duck is flying in the sky.

Duck is swimming in the water.

3. Multilevel Inheritance

Problem Statement

Write a Java program to demonstrate Multilevel Inheritance, where Puppy inherits from Dog, and Dog inherits from Animal.

Steps

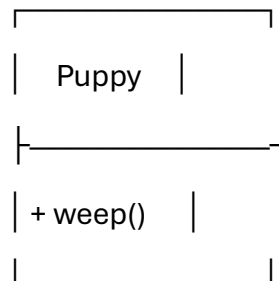
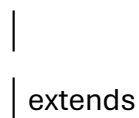
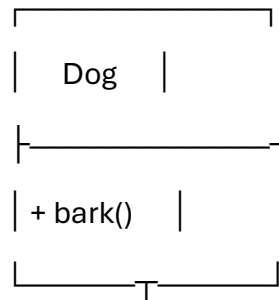
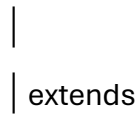
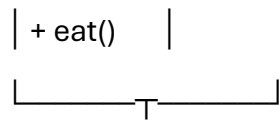
- Create a parent class Animal.
- Define the eat() method.
- Create a Dog class extending Animal.
- Define the bark() method.
- Create a Puppy class extending Dog.
- Define the weep() method.
- Create an object of Puppy.
- Access methods from all three levels.

Algorithm

1. Start.
2. Define class Animal with eat().
3. Define class Dog extending Animal.
4. Define bark() inside Dog.
5. Define class Puppy extending Dog.
6. Define weep() inside Puppy.
7. Create a Puppy object.
8. Call puppy.eat().
9. Call puppy.bark().
10. Call puppy.weep().
11. Stop.

UML Diagram





Code

// File: MultilevelInheritance.java

// First level: Parent class

```
class Animal {
    void eat() {
        System.out.println("Animal eats food.");
    }
}
```

```
}
```

// Second level: Dog inherits from Animal

```
class Dog extends Animal {
```

```
    void bark() {
```

```
        System.out.println("Dog barks.");
```

```
    }
```

```
}
```

// Third level: Puppy inherits from Dog.

// Since Dog inherits from Animal, Puppy can also access

// methods of Animal.

// This is called Multilevel Inheritance.

```
class Puppy extends Dog {
```

```
    void weep() {
```

```
        System.out.println("Puppy is weeping.");
```

```
    }
```

```
}
```

```
public class MultilevelInheritance {
```

```
    public static void main(String[] args) {
```

```
        Puppy puppy = new Puppy();
```

```
        // Inherited from Animal (grandparent class)
```

```
        puppy.eat();
```

```
// Inherited from Dog (parent class)

puppy.bark();

// Method belonging to Puppy

puppy.weep();

}

}
```

Output

Animal eats food.

Dog barks.

Puppy is weeping.

4. Hierarchical Inheritance

Problem Statement

Write a Java program to demonstrate Hierarchical Inheritance, where multiple child classes (Dog and Cat) inherit from the same parent class Animal.

Steps

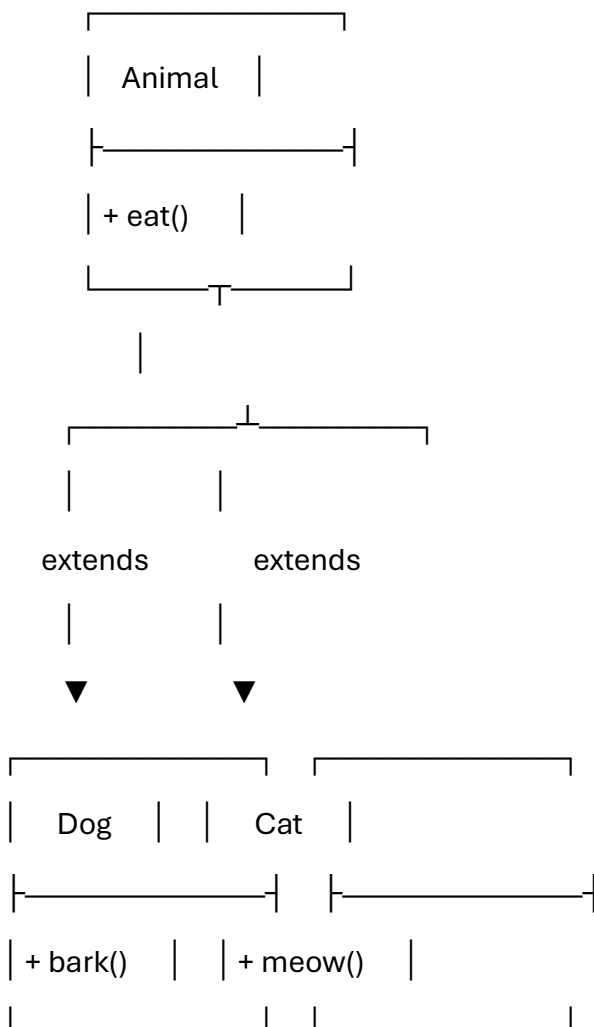
- Create a parent class Animal.
- Define the eat() method.
- Create a Dog class extending Animal.
- Define the bark() method.
- Create a Cat class extending Animal.
- Define the meow() method.
- Create objects of both Dog and Cat.
- Call their inherited and individual methods.

Algorithm

1. Start.

2. Define class Animal with eat().
3. Define class Dog extending Animal.
4. Define bark() inside Dog.
5. Define class Cat extending Animal.
6. Define meow() inside Cat.
7. Create Dog and Cat objects.
8. Call dog.eat() and dog.bark().
9. Call cat.eat() and cat.meow().
10. Stop.

UML Diagram



Code

// File: HierarchicalInheritance.java

// Parent class

```
class Animal {  
    void eat() {  
        System.out.println("Animal eats food.");  
    }  
}
```

// Dog inherits from Animal

```
class Dog extends Animal {  
    void bark() {  
        System.out.println("Dog barks.");  
    }  
}
```

// Cat also inherits from Animal.

// Multiple child classes inherit from the same parent class.

// This is called Hierarchical Inheritance.

```
class Cat extends Animal {  
    void meow() {  
        System.out.println("Cat meows.");  
    }  
}
```

```
public class HierarchicalInheritance {  
    public static void main(String[] args) {
```

```
Dog dog = new Dog();
Cat cat = new Cat();

System.out.println("--- Dog Behavior ---");

// Shared parent method
dog.eat();

// Dog's own method
dog.bark();

System.out.println("\n--- Cat Behavior ---");

// Shared parent method
cat.eat();

// Cat's own method
cat.meow();
}
}
```

Output

--- Dog Behavior ---

Animal eats food.

Dog barks.

--- Cat Behavior ---

Animal eats food.

Cat meows.

5. Hybrid Inheritance

Problem Statement

Write a Java program to demonstrate Hybrid Inheritance by combining class inheritance and interface implementation. A Bat class should inherit from Animal and implement the Flyable interface.

Steps

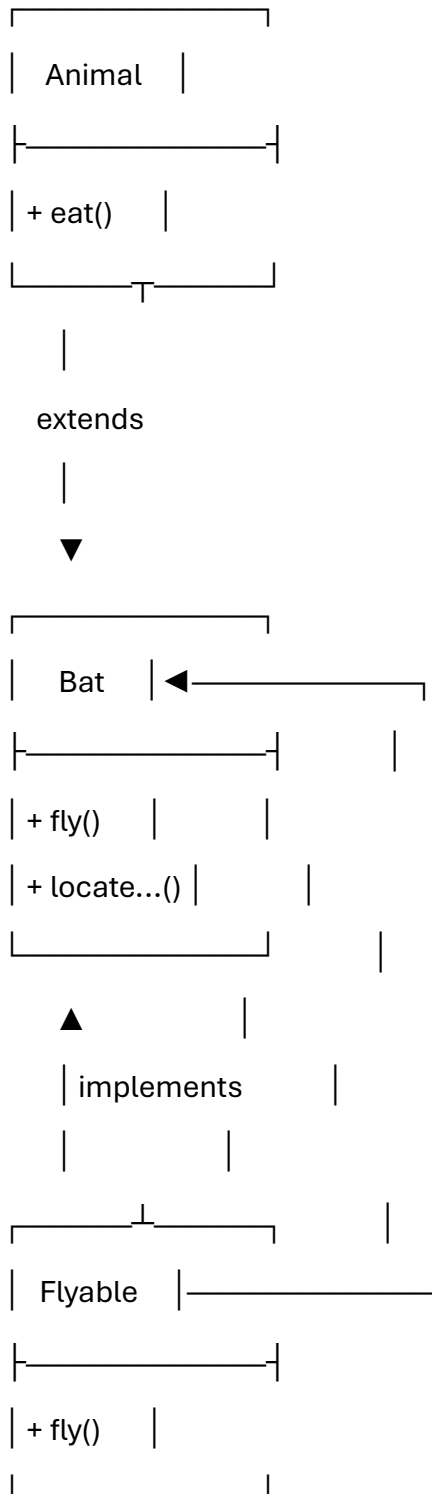
- Create a parent class Animal.
- Define the eat() method.
- Create an interface Flyable.
- Declare the fly() method.
- Create a Bat class extending Animal.
- Make Bat implement Flyable.
- Implement the fly() method.
- Define locateWithSound() inside Bat.
- Create a Bat object.
- Call all available methods.

Algorithm

1. Start.
2. Define class Animal with eat().
3. Define interface Flyable with fly().
4. Define class Bat extending Animal.
5. Implement Flyable in Bat.
6. Define fly() in Bat.
7. Define locateWithSound() in Bat.
8. Create a Bat object.
9. Call eat().
10. Call fly().
11. Call locateWithSound().

12. Stop.

UML Diagram



Code

```
// File: HybridInheritance.java
```

```
// Parent class
```

```
class Animal {  
    void eat() {  
        System.out.println("Animal eats food.");  
    }  
}
```

```
// Interface
```

```
interface Flyable {  
    void fly();  
}
```

```
// Bat extends a class AND implements an interface.
```

```
//
```

```
// It inherits from Animal through class inheritance
```

```
// and obtains the fly() behavior through Flyable.
```

```
//
```

```
// Combining different inheritance mechanisms is known
```

```
// as Hybrid Inheritance.
```

```
class Bat extends Animal implements Flyable {
```

```
    @Override
```

```
    public void fly() {  
        System.out.println("Bat flies at night.");  
    }  
}
```

```
}

void locateWithSound() {
    System.out.println("Bat uses echolocation.");
}
}
```

```
public class HybridInheritance {
    public static void main(String[] args) {

        Bat bat = new Bat();

        // Inherited from Animal class
        bat.eat();

        // Implemented from Flyable interface
        bat.fly();

        // Method belonging to Bat
        bat.locateWithSound();
    }
}
```

Output

Animal eats food.

Bat flies at night.

Bat uses echolocation.